GATE CS — Algorithms

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
Graph algorithms: MST, shortest path, BFS/DFS	3.7	Tier 1
Asymptotics & recurrences	1.5	Tier 2
Sorting, searching, divide & conquer	0.9	Tier 3
Hashing	0.5	Tier 3
Greedy	0.3	Tier 3
Dynamic programming	0.3	Tier 3

Algorithms carries about **7.3 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Algorithms.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Graph Algorithms: MST, Shortest Paths, BFS/DFS

Representation and traversals

- Adjacency matrix: $O(V^2)$ space, $O(1)$ edge test. Adjacency list: $O(V+E)$ space, best for sparse graphs.
- **BFS** (queue): $O(V+E)$; finds shortest paths by number of edges in unweighted graphs; edge types in undirected BFS: tree and cross only. **DFS** (stack/recursion): $O(V+E)$.
- DFS edge classification: tree, back (to an ancestor), forward, cross. A directed graph has a cycle iff DFS finds a back edge. An undirected graph has a cycle iff DFS finds a non-parent edge to a visited vertex.
- **Topological order** of a DAG: reverse of DFS finishing times (or repeatedly remove in-degree-0 vertices). Exists iff the graph is acyclic; not unique in general.
- **Strongly connected components** (Kosaraju or Tarjan): $O(V+E)$. Articulation points and bridges via DFS low-link values.
- Bipartite test: BFS/DFS 2-colouring.

Minimum spanning tree

- **Cut property:** the lightest edge crossing any cut belongs to some MST. **Cycle property:** the heaviest edge on a cycle (unique heaviest) is in no MST.
- **Kruskal:** sort edges, add if it joins different components (union–find): $O(E \log E) = O(E \log V)$. **Prim:** grow from a vertex using a priority queue: $O(E \log V)$ with a binary heap, $O(E + V \log V)$ with a Fibonacci heap, $O(V^2)$ with an array.
- If all edge weights are distinct, the MST is unique. The MST has $V - 1$ edges. The MST also minimises the maximum edge on the path between two vertices (bottleneck property).
- Adding a new edge to an MST: add it, find the cycle, and delete the heaviest edge on that cycle. Increasing or decreasing weights uniformly (or squaring positive weights) keeps the MST unchanged.
- Number of MSTs: when weights tie, count choices per weight class; for K_n with all equal weights it is n^{n-2}.

Shortest paths

Algorithm	Handles	Time
BFS	unweighted	$O(V+E)$
Dijkstra	non-negative weights, single source	$O((V+E) \log V)$ with binary heap; $O(V^2)$ with array
Bellman–Ford	negative edges, detects negative cycles	$O(VE)$
Floyd–Warshall	all pairs, negative edges without negative cycles	$O(V^3)$
DAG shortest path	topological order, any weights	$O(V+E)$

- Dijkstra fails with negative edges; adding a constant to every edge also changes the shortest path (paths with more edges are penalised more).
- Dijkstra finalises a vertex when it is extracted from the queue. Modified-Dijkstra questions (extra state such as number of edges used, or minimising the maximum edge) need a state-augmented graph.
- Floyd–Warshall: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])` with k as the outer loop.
- Single-pair shortest path is no easier asymptotically than single-source. Number of shortest

paths: count along the shortest-path DAG.

> **Trap:** Prim and Dijkstra look similar but Prim compares the edge weight, while Dijkstra compares the total distance from the source.

2. Asymptotics & Recurrences

- f = O(g): f ≤ c·g eventually. Ω: lower bound. Θ: both. o and ω are strict versions. Compare by limits of f/g: 0 → o, finite non-zero → Θ, ∞ → ω.
- Growth order (slow → fast): $1 < \log\log n < \log n < (\log n)^k < n^\varepsilon < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$. Also $2^{\sqrt{n}}$ beats every polynomial; $(\log n)^{\log n}$ grows faster than any polynomial; $n^{\log n}$ lies between polynomials and exponentials.
- $\log(n!) = \Theta(n \log n)$ (Stirling). $\Sigma\, 1/i = \Theta(\log n)$. $\Sigma\, i = \Theta(n^2)$. $\Sigma\, 2^i$ $(i \le k) = 2^{k+1} - 1$. $\Sigma\, i \cdot 2^i$ is $\Theta(k\, 2^k)$.
- Base of the log does not matter inside O(·), but exponent bases do: $2^n = O(3^n)$ but $3^n \neq O(2^n)$.

Solving recurrences

Master theorem: T(n) = aT(n/b) + f(n), a ≥ 1, b > 1. Compare f(n) with n^(log_b a):
• f(n) = O(n^(log_b a − ε)) ⇒ T(n) = Θ(n^(log_b a))
• f(n) = Θ(n^(log_b a)·log^k n) ⇒ T(n) = Θ(n^(log_b a)·log^(k+1) n)
• f(n) = Ω(n^(log_b a + ε)) and regularity a·f(n/b) ≤ c·f(n), c < 1 ⇒ T(n) = Θ(f(n))

- Examples: T = 2T(n/2) + n → n log n (merge sort); T = T(n/2) + 1 → log n (binary search); T = 2T(n/2) + 1 → n; T = 7T(n/2) + n² → n^2.81 (Strassen); T = 4T(n/2) + n → n².
- Not covered by the Master theorem: T = 2T(n/2) + n/log n → Θ(n log log n) (sum level by level). T = T(n−1) + n → n²/2. T = 2T(n−1) + 1 → $2^n - 1$. T = T(n/3) + T(2n/3) + n → Θ(n log n) (recursion tree).
- T(n) = T(√n) + 1 → Θ(log log n). T(n) = T(n−1) + log n → Θ(n log n).
- Recursion tree: sum the work per level times the number of levels; use for unequal splits.

Analysing loops

- Nested loops with i doubling: `for(i=1;i<n;i*=2)` runs log n times. `for(i=n;i>0;i/=2)` likewise. `for(i=1;i*i<=n;i++)` runs √n times.
- Dependent inner loop `for j = 1 to i` gives $n(n+1)/2 = \Theta(n^2)$. Outer n, inner log n → n log n.

3. Sorting, Searching, Divide & Conquer

Sort	Best	Average	Worst	Stable	In-place
Bubble / selection / insertion	n (bubble, insertion with a flag)	n^2	n^2	bubble, insertion yes; selection no	yes
Merge sort	n log n	n log n	n log n	yes	no (O(n) extra)
Quick sort	n log n	n log n	n^2 (sorted input with first/last pivot)	no	yes
Heap sort	n log n	n log n	n log n	no	yes
Counting sort	O(n + k) for keys in 0…k			yes	no
Radix sort	O(d(n + k))			yes	no

- Comparison-based sorting needs $\Omega(n \log n)$ comparisons in the worst case (decision tree has $n!$ leaves, height $\geq \log_2 n!$).
- Insertion sort cost = n + number of inversions; it is fast on nearly sorted input. Swaps in bubble sort = number of inversions.
- Merge sort comparisons: at most $n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$. For n = 8: 17. Merging sorted lists of sizes m and n needs at most $m + n - 1$ comparisons.
- Quick sort recurrence: balanced split $2T(n/2) + n$; worst split $T(n-1) + n$. Randomised pivot gives expected O(n log n). Partition itself is O(n).
- **Binary search:** $\lfloor \log_2 n \rfloor + 1$ comparisons worst case on a sorted array. Linear search: n. Binary search on a linked list is not O(log n).
- Max and min together: $\lceil 3n/2 \rceil - 2$ comparisons. Second largest: $n + \lceil \log_2 n \rceil - 2$ (tournament). Median of medians: worst-case O(n) selection. Quick-select: expected O(n), worst $O(n^2)$.
- Divide-and-conquer examples: closest pair O(n log n), Strassen O(n^2.81), maximum subarray O(n log n), counting inversions with merge sort O(n log n), Karatsuba O(n^1.585).
- Minimum number of comparisons to sort 3, 4, 5 elements: 3, 5, 7.

4. Hashing

- Load factor $\alpha = n/m$. Chaining: expected search time $\Theta(1 + \alpha)$ under simple uniform hashing. Open addressing needs $\alpha \leq 1$; expected probes $\approx 1/(1-\alpha)$ for an unsuccessful search.
- **Linear probing:** $h(k, i) = (h(k) + i) \bmod m$ — suffers primary clustering. **Quadratic:** $(h(k) + c_1 i + c_2 i^2)$ — secondary clustering. **Double hashing:** $(h_1(k) + i \cdot h_2(k)) \bmod m$; h_2 must be non-zero and relatively prime to m to visit every slot.
- Deletion in open addressing needs a tombstone marker, not an empty slot, or later searches stop too early.
- Worst-case search in a hash table is $O(n)$ if all keys collide. Number of keys hashing to the same slot can be forced by the choice of h and the inserted keys.
- Typical hash functions: division $h(k) = k \bmod m$ (choose m prime, not a power of 2), multiplication method, mid-square.
- Insertion sequence questions: apply the hash function and probe rule key by key and write the table.

> **Tip:** For “where does the last key land” questions, compute the probe sequence only for that key, but make sure you have the final state of earlier keys right first.

5. Greedy Algorithms

- A greedy algorithm makes the locally best choice at each step; correct when the problem has the greedy-choice property and optimal substructure. Prove it by an exchange argument.
- **Activity selection** (maximum non-overlapping): sort by finish time. **Fractional knapsack**: sort by value/weight; greedy is optimal (0/1 knapsack is not — use DP).
- **Huffman coding:** repeatedly merge the two lowest-frequency nodes. Total weighted length = sum of the internal node weights. Optimal prefix code; with n symbols there are n − 1 merges. Complexity O(n log n).
- **Job sequencing with deadlines:** sort jobs by profit (descending), place each in the latest free slot at or before its deadline; unit-time jobs.
- **Interval scheduling / minimum platforms / merging files (optimal merge pattern):** merge the two smallest each time, like Huffman.
- Prim, Kruskal and Dijkstra are greedy. Coin change with denominations like 1, 5, 10, 25 works greedily, but not for every coin system (e.g. {1, 3, 4} for 6).
- Greedy fails for 0/1 knapsack, longest path, travelling salesperson. When a counter-example exists, use DP or search.

6. Dynamic Programming

- Use when subproblems overlap and the problem has optimal substructure. Top-down (memoisation) or bottom-up (tabulation). Define the state, the recurrence, the base case and the evaluation order.
- **Longest common subsequence:** L[i][j] = L[i−1][j−1] + 1 if X[i] = Y[j], else max(L[i−1][j], L[i][j−1]). Time O(mn). **Edit distance**: min of insert, delete, replace (+1) or match (+0).
- **0/1 knapsack:** V[i][w] = max(V[i−1][w], v_i + V[i−1][w − w_i]); O(nW) (pseudo-polynomial). **Unbounded** knapsack/coin change: V[w] = max over items.
- **Longest increasing subsequence:** $O(n^2)$ DP or O(n log n) with patience sorting. **Matrix-chain multiplication:** M[i][j] = min over k of M[i][k] + M[k+1][j] + p_{i−1}p_k p_j; $O(n^3)$.
- **Counting problems:** ways to climb n stairs, grid paths (blocked cells), partitions, coin-change counting (loop over coins first to count combinations, not permutations).
- **Bellman–Ford**, **Floyd–Warshall** and optimal BST are DP; longest path in a DAG is DP in topological order.
- Space optimisation: keep only the previous row (O(n) space), but then you cannot reconstruct the sequence without extra work.
- Subset-sum (O(nS)) and partition problems are DP; TSP via bitmask DP takes $O(n^2\,2^n)$.

> **Trap:** “Non-standard” DP questions change the recurrence slightly (a forbidden pair, a cost per transition, a circular array). Write the recurrence from the new rule instead of recalling a template.

7. Exam strategy & revision checklist

- Study order: graph algorithms → DP and greedy → recurrences and asymptotics → sorting/searching → hashing. Recent papers lean toward harder graph and DP questions.
- Know the complexity of each algorithm, the data structure used, and a one-line reason.
- ☐ Apply Master theorem and recursion trees. ☐ Run Kruskal, Prim, Dijkstra by hand. ☐ Fill LCS and knapsack tables. ☐ Build a Huffman tree. ☐ Insert keys into a hash table using each probing scheme. ☐ Redo `GATE_CS_Algorithms.pdf` past questions.